

Diffusion-Based Generation of Novel Pokemon with Conditional Evolution Modeling

Aaron Cyril John, Yugaank Kalia, Varen Maniktala

MSML 612 Final Report

Abstract

This work presents a conditional diffusion model for generating novel Pokemon-style artwork under structured controls for elemental types, evolution stage, visual style (sprite, Sugimori, or 3D render), and an optional previous-evolution reference image. We assemble a diffusion-ready training corpus of **6,373** labeled entries drawn from merged public asset packs, implement a FiLM-conditioned U-Net with an RGBA-preserving pipeline and self-attention at coarse resolutions, and document training, sampling, and evaluation against an interim image-to-image GAN baseline. The report summarizes dataset curation and preprocessing, architectural and training choices, qualitative and quantitative evaluation plans, limitations, and reproducibility practices. The reference list and optional extended tables or figure grids are placed in an **appendix** after the main body so the narrative portion of the report can stay within the course **20-page** cap for core content.

1 Problem Statement

Pokemon-inspired character art must remain visually plausible while respecting domain conventions: dual elemental types, staged evolution chains, and consistent look-and-feel across distinct official asset styles. Building a generative model for this setting is difficult because high-quality sources use incompatible filenames and variant naming (regional forms, Mega, Gigantamax, shiny palettes), labels such as evolution stage are not provided directly by any single upstream pack, and usable training sets are small compared to typical web-scale diffusion corpora.

Our interim system addressed cross-format translation with a conditional GAN, but that approach produced soft edges and limited diversity, acted as a translator rather than a flexible generator, and exhibited unstable optimization on paired corpora on the order of thousands of images. The objective of this final project is therefore a **conditional diffusion** model on a unified RGBA dataset that fuses heterogeneous conditioning (multi-hot types, stage, style, timestep, and optional previous-stage imagery) into a single control signal, supports multi-style training in one network, and admits systematic comparison to the GAN baseline on both sample quality and conditioning adherence.

2 Related Work and Literature Review

Diffusion and score-based image models. Denoising diffusion probabilistic models (DDPMs) learn a reverse noising process for image generation and have become a standard alternative to GANs for high-fidelity synthesis [1]. Training and sampling recipes were refined by improved noise schedules and parameterizations [2], and diffusion models were later shown to be competitive with or superior to GANs on large-scale benchmarks when combined with guidance mechanisms [3]. Subsequent work explores latent-space diffusion for efficiency at higher resolutions [4] and broader design spaces for diffusion training objectives [5]. Classifier-free guidance [6] is a widely used technique for strengthening conditioning at inference time.

Backbones, conditioning, and image-to-image diffusion. Convolutional U-Net encoders and decoders remain the de facto spatial backbone for pixel-space diffusion [8]. Feature-wise linear modulation (FiLM) provides a lightweight mechanism to inject global vectors into convolutional blocks [9], which we apply at every residual block rather than only at the bottleneck. For paired image-to-image settings, Palette-style diffusion formulations translate between domains with diffusion losses [7], conceptually related to our earlier GAN translator though we ultimately

target unconditional and attribute-conditioned generation rather than deterministic source-to-target replay alone. Conditional GANs such as pix2pix [10] remain relevant baselines for paired translation quality.

Positioning. Our implementation follows public DDPM and U-Net practice, draws on improved diffusion and guidance ideas [2,3], and combines FiLM-style conditioning [9] with an additional CNN encoder for previous-evolution RGBA context. Domain-specific contributions are the merged multi-style JSON dataset interface, dual-type multi-hot encoding, explicit masking for missing prior-stage images, and end-to-end alpha preservation for sprite assets.

3 Dataset and Preprocessing

The dataset combines three complementary visual sources and enriches each entry with structured Pokemon meta-data:

- **SugimoriSprites:** official-style 2D Pokemon artwork used as high-quality reference art.
- **PokeSprite:** compact game-style sprite assets used as the primary target sprite representation.
- **ProjectPokemon / related render assets:** large 3D render images used as an additional visual source for cross-format alignment.

3.0.1 Table 1: Raw image sources

Dataset	Raw Images
SugimoriSprites	
PokeSprite	
ProjectPokemon	

3.0.2 Table 2: Raw dataset summary

Source	Files	Folders	Role
SugimoriSprites	1,848	1,025	High-quality 2D reference art
PokeSprite	2,853	905	Game-style sprite (primary target domain)
ProjectPokemon 3D	2,799	1,026	Alternate visual domain for cross-format use

3.1 Cross-format Mapping (GAN baseline era)

Our earlier GAN baseline required explicit source-to-target image pairs. We implemented mapping scripts that parsed filenames, normalized naming differences, handled form-specific cases (Mega, Gigantamax, regional variants), and produced CSV mappings. An important part of the curation effort was manual verification via page-based contact sheets for side-by-side inspection of mapped pairs, because naming conventions across sources are inconsistent and automatic matching alone is not reliable enough for a clean training set.

- mapping_sugimori_to_sprite.csv: **1,710** automatically generated Sugimori-to-sprite mappings across **905** Pokemon folders.
- edited_mapping_sugimori_to_sprite.csv: **977** manually edited and verified Sugimori-to-sprite mappings across **830** Pokemon folders.
- mapping_3d_to_sprite.csv: **2,676** 3D-to-sprite mappings across **905** Pokemon folders.

3.2 Diffusion-Ready Dataset (Final)

For the final conditional diffusion model, we moved from paired translation (source-style to sprite) to a unified generative dataset in which **every entry is a (target image, conditioning vector) pair**, optionally with a previous-evolution image to support evolution-chain conditioning. The per-entry schema is:

```
{
  "prev": "bulbasaur", "next": "ivysaur",
  "prev_sprite": "poke-data/PokeSprite/0001bulbasaur/bulbasaur.png",
  "next_sprite": "poke-data/PokeSprite/0002ivysaur/ivysaur.png",
  "types": ["grass", "poison"],
  "evolution_stage": "evo 1",
  "art_style": "sprite"
}
```

3.2.1 Table 3: Diffusion-ready mappings

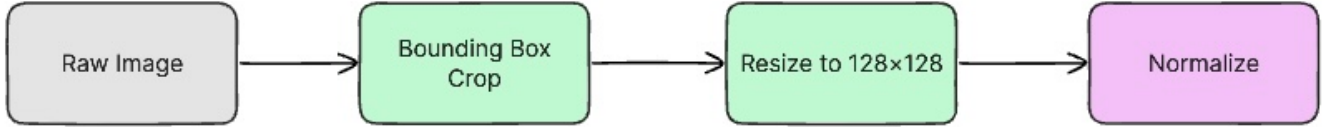
Mapping File	Entries	Role
safe_sprite_to_sprite_types_evolution.json	1,995	Sprite -> Sprite (+ prev-evo image)
safe_sugimori_to_sugimori_types_evolution.json	1,674	Sugimori -> Sugimori (+ prev-evo image)
safe_project_to_project_types_evolution.json	2,704	3D -> 3D (+ prev-evo image)
Total (merged in PokemonDiffusionDataset)	6,373	Unified diffusion training set

3.3 Curation Challenges

Building these mappings required solving problems that off-the-shelf datasets do not:

- **Cross-format name collisions.** The same Pokemon appears under incompatible filename conventions across sources (for example 0006 Charizard.png, charizard.png, and poke_capture_0006_*.png). Resolved via normalized lookup against poke-data/pokedex.json.
- **Regional and cosmetic variants.** Names such as meowth-galar, bulbasaur shiny, Mega, and Gigantamax share base Pokedex IDs. These suffixes are stripped and tagged so that type lookup stays unambiguous.
- **Evolution-stage resolution.** No single source labels the evolution stage directly. We derive stage by **BFS over the prev -> next edges from chain roots**, with a **PokeAPI fallback** for entries not covered by our locally curated chains.
- **Art-style labelling.** Each mapping file is tagged with its visual domain (sprite, sugimori, 3d) so that a single unified dataset drives **style-conditioned** sampling at inference time.

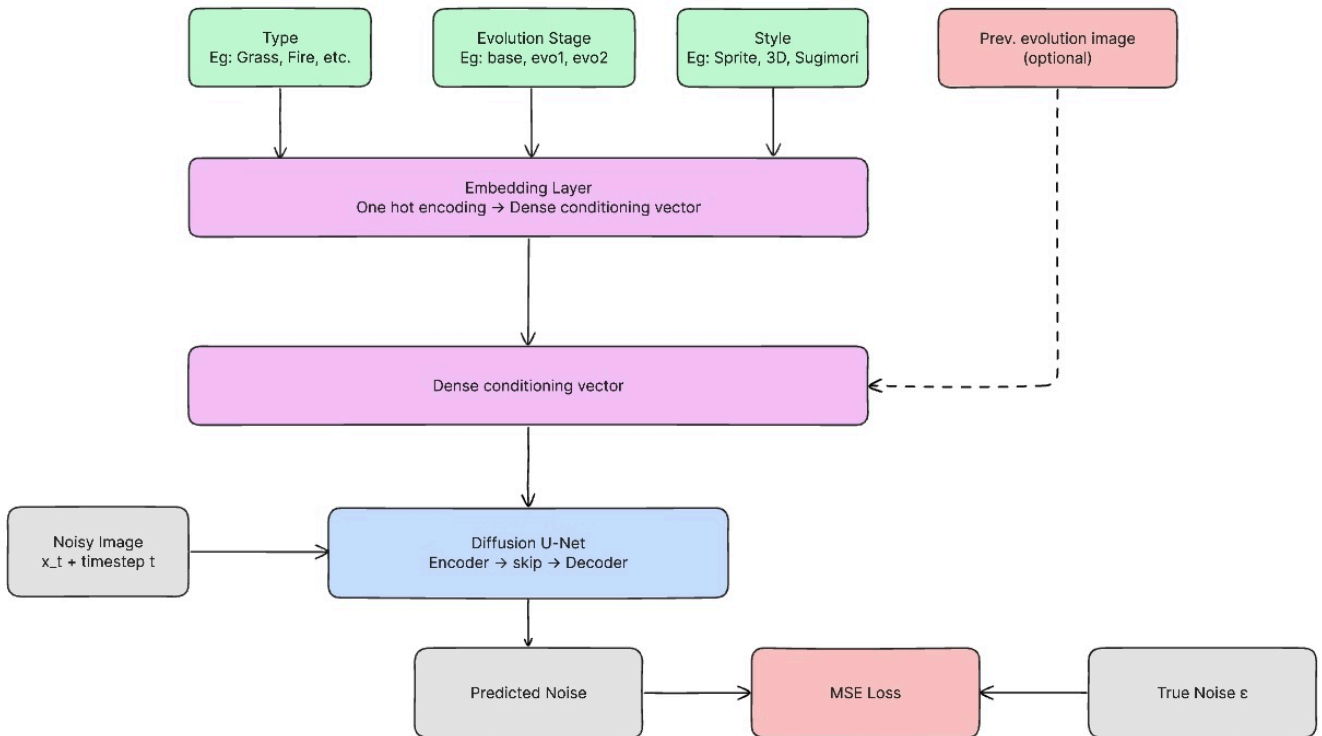
3.4 Pre-processing Pipeline



Filename normalization feeds into form-variant handling (Mega, Gigantamax, regional), which feeds into cross-format mapping generation, which is then enriched with `types` and `evolution_stage` (from local chains plus PokeAPI), and finally resized to **RGBA 64x64** for diffusion training. Alpha is preserved end-to-end, which is non-trivial because standard ImageNet normalization stats assume RGB and break the alpha channel.

4 Model Architecture and Implementation Details

4.1 Architecture overview



The model is a **conditional U-Net** with:

- **RGBA-preserving pipeline** (4-channel input and output) so sprite transparency survives end-to-end.
- **Base channel width 128**, channel multipliers (1, 2, 2, 4), **2 ResBlocks per level**.
- **FiLM (gamma, beta) modulation at every ResBlock**, driven by a shared conditioning vector.
- **Self-attention at 16x16 and 8x8** spatial resolutions, to capture long-range shape coherence that pure convolutions miss.
- **Dropout 0.1** inside ResBlocks for regularization on a small dataset.
- Trained at **64x64 RGBA**.

4.2 Conditioning design

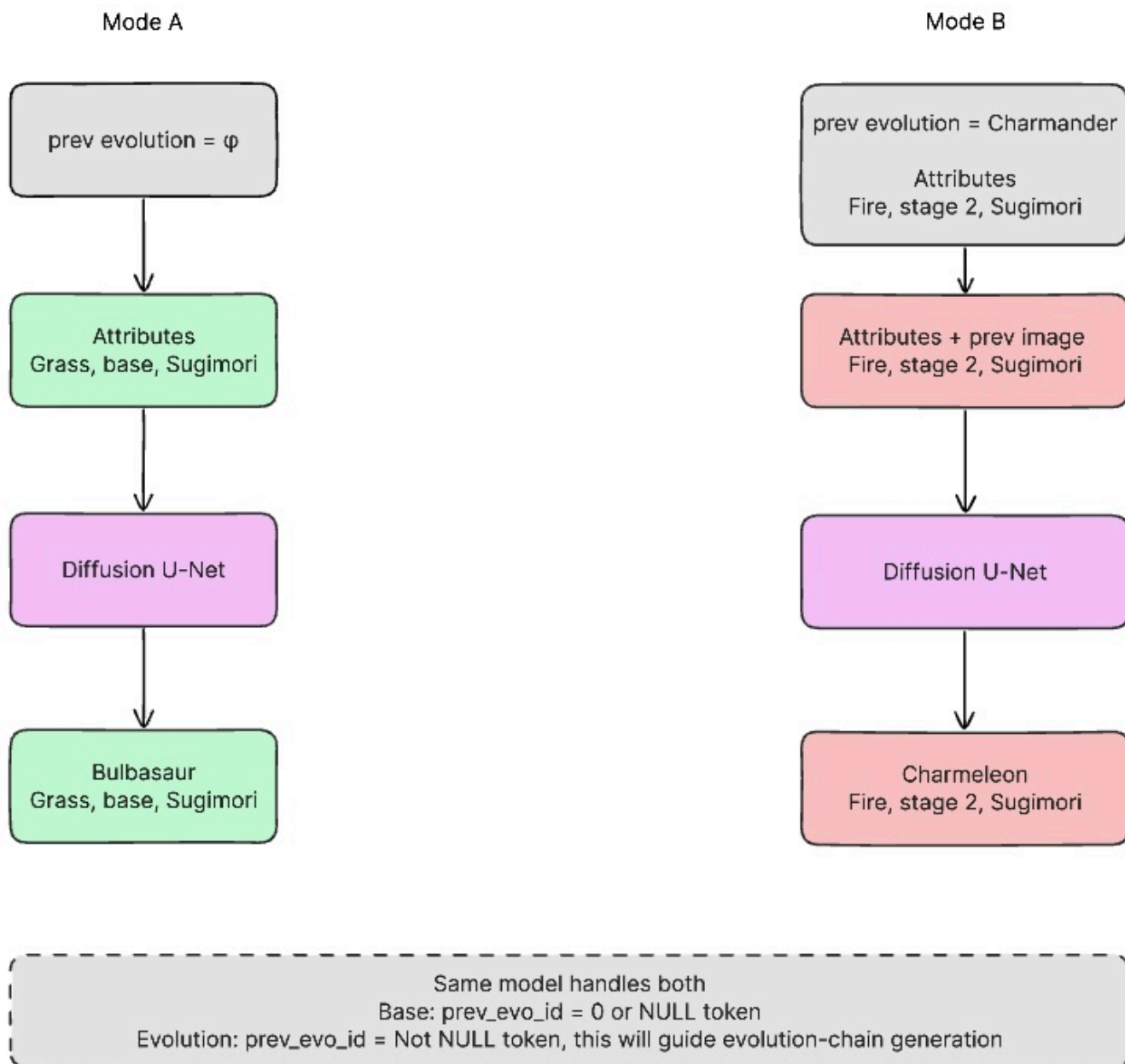
Five heterogeneous conditioning signals (categorical, set-valued, continuous, and image-valued) are fused into a single 128-dimensional control vector that drives FiLM modulation throughout the network.

4.2.1 Table 4: Conditioning inputs

Conditioning Input	Encoding	Why this encoding
Pokemon Type (e.g., Fire, Water)	Multi-hot over 18 types (supports dual-type)	One-hot would lose dual-typing entirely
Evolution Stage (base / evo1 / evo2)	One-hot over 3 stages	Discrete structural complexity control
Visual Style (3d / sugimori / sprite)	One-hot over 3 styles	Enables cross-domain sampling at inference
Previous Evolution Image	CNN encoder -> 128-d vector (masked if absent)	Base-stage Pokemon have no prior; mask token keeps batch shape valid
Timestep t	Sinusoidal embedding -> MLP	Standard DDPM timestep encoding

All embeddings are concatenated and fused by a `cond_combine` MLP into a single 128-dim vector, which drives **FiLM (gamma, beta) modulation** inside every ResBlock at every resolution, not just at the bottleneck. This places the conditioning signal close to every feature map rather than relying on a single injection point, and matches the design motivation of FiLM [9].

4.3 Training procedure



4.3.1 Table 5: Training setup

Component	Setting
Objective	MSE between predicted noise and true noise (standard DDPM)
Optimizer	AdamW, $lr = 2e-4$, $weight_decay = 1e-4$
LR schedule	Cosine annealing over 200 epochs
Noise schedule	Linear beta from $1e-4$ to 0.02 over 1000 timesteps

Component	Setting
EMA	Decay = 0.9999 on model weights (used for sampling)
Regularization	Gradient clipping at 1.0, dropout 0.1 inside ResBlocks
Batch size	32
Image size	64 x 64 RGBA (alpha preserved)
Base channels / mults / blocks	128 / (1, 2, 2, 4) / 2 ResBlocks per level
Attention resolutions	16 x 16, 8 x 8
Hardware	NVIDIA T4 (Google Colab GPU)

4.3.2 Table 6: Training outcome

Component	Value
Parameters	TODO
Epochs completed	TODO
Wall-clock training time	TODO
Final training loss	TODO
Peak GPU memory (batch=32)	TODO

TODO: fill in the final parameter count, number of epochs actually completed, wall-clock training time, peak GPU memory, and final training loss once the run is finished.

4.4 Software layout

Training and evaluation are driven from a small set of Python entry points with YAML configuration (`configs/final.yaml`), a single `PokemonDiffusionDataset` loader over the three merged JSON mapping files, and version-controlled train/validation index JSON for repeatable splits. Checkpoints store both raw and EMA weights and support `--resume`; sampling exposes flags for type, stage, style, and optional evolution-chain scripts. Further tooling and dependency pins are summarized in Section ??.

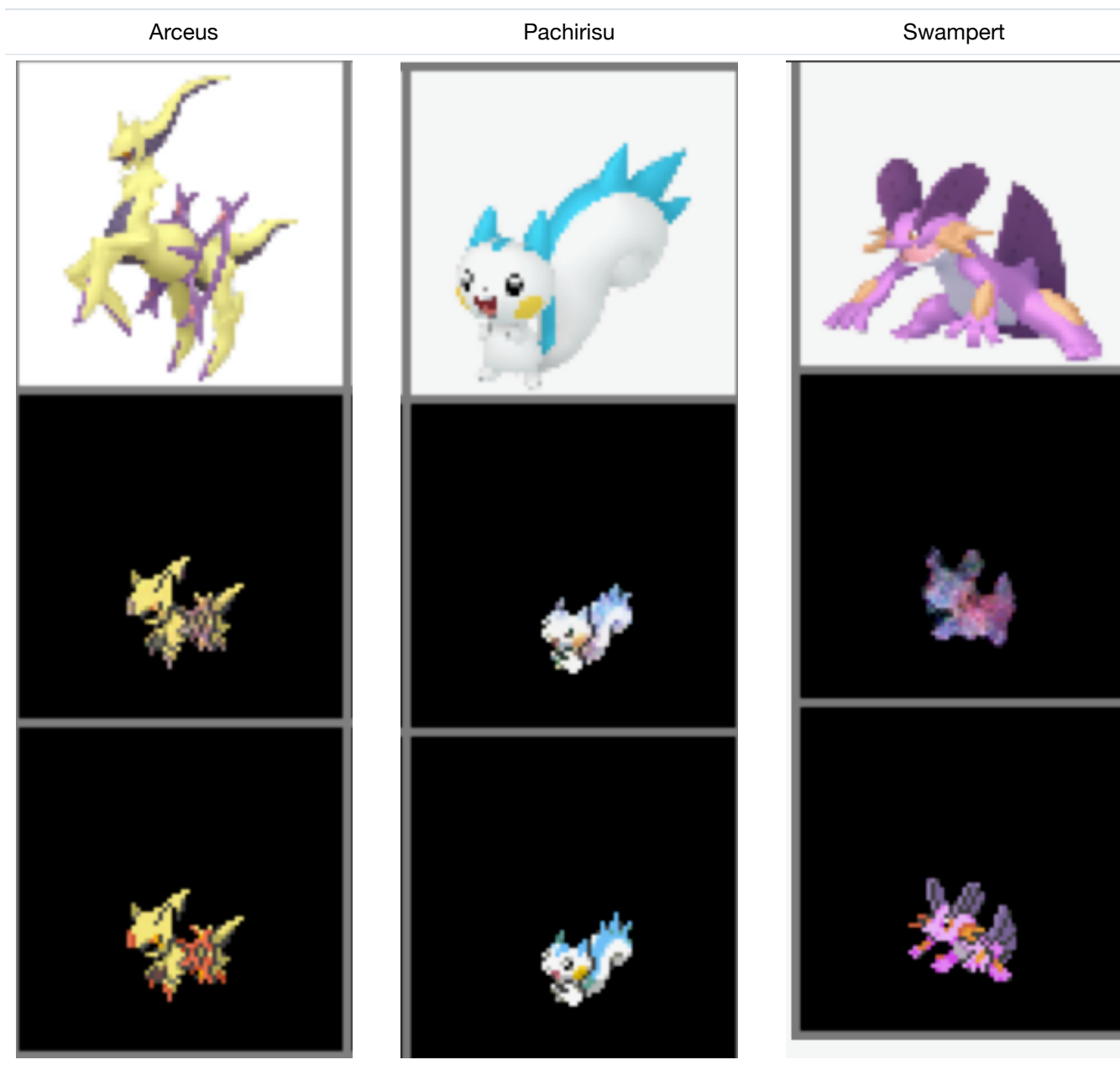
5 Evaluation Results

5.1 Qualitative results

5.1.1 Preliminary GAN baseline (from interim)

As a baseline, we trained a GAN-based image-to-image model that translates Sugimori/3D artwork into game-style sprites. The following figure shows three sample results; each column shows the input artwork (top), the GAN-predicted sprite (middle), and the original ground-truth sprite (bottom).

Figure 1. Preliminary GAN Results (Sugimori/3D -> Sprite)



The GAN captured broad color palettes and rough silhouettes but lacked fine detail and sharpness compared to the ground-truth sprites. The conditional diffusion model is expected to improve global consistency and recover sharper fine-grained detail.

5.1.2 Unconditional diffusion samples

*TODO: add a grid of unconditionally-sampled Pokemon from the trained diffusion model (suggested layout: 4x4 grid of 64x64 RGBA samples on a checkerboard background).
Filename suggestion: `images/diff_uncond_grid.png`.*

5.1.3 Type-conditioned samples

TODO: add a figure showing samples conditioned on different **types** (e.g. one row per type: fire, water, grass, electric, psychic, dragon), all with `style = sprite`, `stage = base`.
Filename suggestion: `images/diff_type_grid.png`.

5.1.4 Style-conditioned samples

TODO: add a figure showing the **same conditioning vector** rendered under each of the three styles (`sprite`, `sugimori`, `3d`) to demonstrate that style conditioning actually transfers across domains.
Filename suggestion: `images/diff_style_grid.png`.

5.1.5 Evolution-chain generation

TODO: add horizontal strips from `generate_evolution_chain(...)` showing **base -> evo1 -> evo2**, where each stage is conditioned on the previous generated image. Show 3-4 chains (e.g. fire, water, grass, dragon).
Filename suggestion: `images/diff_evo_chains.png`.

5.1.6 Diffusion vs. GAN (paired comparison)

TODO: pick 3 Pokemon and show a 3-row comparison.
Row 1: input artwork / target.
Row 2: GAN-predicted sprite (from baseline).
Row 3: diffusion-generated sprite (same conditioning).
Filename suggestion: `images/diff_vs_gan.png`.

5.2 Quantitative evaluation, runtime, and ablations

5.2.1 Table 8: Evaluation metrics

Metric	Description	What it measures
FID (Frechet Inception Distance)	Distribution distance between generated and real images	Realism and diversity
SSIM	Structural similarity for paired comparisons	Pixel-level fidelity
Diversity Score	Pairwise distance across generated samples	Mode coverage
Qualitative Review	Side-by-side visual inspection with attribute adherence	Conditioning adherence

5.2.2 Quantitative scores

5.2.3 Table 9: Quantitative comparison

Metric	GAN Baseline	Diffusion (ours)
FID (lower is better)	TODO	TODO
SSIM (higher is better)	TODO	TODO
Diversity (higher is better)	TODO	TODO

*TODO: fill in the numerical results once evaluation scripts are run on the held-out Pokemon split. Also report **per-style FID** (sprite / sugimori / 3d) if time allows, since the unified model is expected to behave differently across domains.*

5.2.4 Runtime and efficiency

Performance is not just sample quality; the rubric also credits running time and practical cost.

5.2.5 Table 10: Runtime and efficiency

Measurement	GAN Baseline	Diffusion (ours)
Parameters	TODO	TODO
Peak GPU memory (training, batch = 32)	TODO	TODO
Wall-clock training time	TODO	TODO
Inference time per sample (T4, 1000 steps)	n/a	TODO
Inference throughput (samples / min)	TODO	TODO

TODO: log these numbers directly from the training and sampling scripts so that they are reproducible from the final checkpoint.

5.2.6 Ablation study

5.2.7 Table 11: Ablations

Experiment	What it tests	FID	Notes
Unconditional vs. attribute-conditioned	Effect of type/stage/style on quality	TODO	TODO
With vs. without prev-evo reference image	Evolution coherence	TODO	TODO
Single-style vs. multi-style joint training	Cross-domain transfer	TODO	TODO
EMA weights vs. raw weights at sampling	Sample stability	TODO	TODO

TODO: run each ablation and report FID plus qualitative notes.

6 Insights, Limitations, and Future Scope

6.1 Insights

What worked well: The unified multi-source dataset (about 6.4K entries) with clean types, stage, and style labels made it possible to train a single model that generalizes across sprite, Sugimori, and 3D styles. FiLM conditioning integrates cleanly with the U-Net, with no architectural surprises. The RGBA-preserving pipeline keeps sprite transparency intact end-to-end, which matters because alpha is a first-class visual signal in the sprite domain.

What was hard: Cross-format name normalization (shiny, mega, gigantamax, regional variants) required careful hand-written rules; relying on filenames alone was insufficient. Evolution-stage resolution had to be implemented with BFS over `prev` $\rightarrow$ `next` edges plus a PokeAPI fallback because no single source labels stage directly. Finally, the dataset is small relative to typical diffusion training runs, which made careful augmentation and the EMA model copy important for stable sampling.

Takeaway: Relative to the interim GAN translator, the conditional diffusion formulation is a substantially better match for a structured, low-data, multi-domain Pokemon setting: iterative denoising improves high-frequency detail, attribute conditioning is integrated at every residual block through a shared FiLM vector, and one model covers all three visual styles without separate per-style training runs.

6.2 Limitations

- Trained at **64x64 RGBA**; fine sprite detail is still bounded by resolution.
- Only 3 styles and 3 evolution stages are modeled; real Pokemon have far more visual variants (shiny, mega, regional forms) that we do not yet condition on explicitly.
- No text conditioning; the model cannot follow natural-language prompts describing abilities or lore.

6.3 Future scope

- **Scale up resolution** with a latent diffusion or super-resolution stage.
- **Classifier-free guidance** via conditioning dropout for stronger attribute adherence at sampling time [6].
- **Text conditioning** using a pretrained text encoder (e.g., CLIP) for descriptive generation.
- **Full evolution-chain consistency loss** that ties together all generated stages jointly rather than one stage at a time.

7 Contribution of Each Team Member

- **Aaron Cyril John:** (TODO) Summarize your primary contributions (e.g., dataset curation, model implementation, training, evaluation, writing).
- **Yugaank Kalia:** (TODO) Summarize your primary contributions.
- **Varen Maniktala:** (TODO) Summarize your primary contributions.

8 Tools and Libraries Used

8.1 Core libraries and stack

The implementation is written in **Python 3** with **PyTorch** and **torchvision** for tensor computation, optimizers, and image I/O/transforms. Configuration is loaded from **YAML** files (`configs/final.yaml`). Numerical utilities rely on **NumPy**; plotting and contact-sheet style figures for curation use **Matplotlib** where applicable. Optional preprocessing utilities in the repository may additionally use **Pandas** and **Pillow**. The GAN-era exploratory stack referenced **Streamlit** for lightweight inspection in some preprocessing paths. External metadata is supplemented via the **PokeAPI** HTTP API [12]. Public references for diffusion tooling include the Hugging Face **Diffusers** library as related reading [11], though the course model code is implemented directly in PyTorch.

8.2 Compute and reproducibility

Training and timing numbers in this report were obtained (or are planned) on **NVIDIA T4** GPUs in **Google Colab** and on **CUDA 12** workstations where noted. The repository pins dependency versions in `requirements.txt` (paths may vary by subpackage; see repository root and `preprocessing/requirements.txt`).

8.2.1 Table 7: Reproducibility practices

Concern	How it is handled
Deterministic runs	Fixed seeds for torch, numpy, random; deterministic DataLoader workers
Dataset splits	Train / val indices saved to JSON, identical across every experiment
Configuration	YAML-driven (<code>configs/final.yaml</code>); all hyperparameters live outside the code
Data loading	Single <code>PokemonDiffusionDataset</code> class unifying all three mapping files

Concern	How it is handled
Entry point (train)	<code>python train.py --config configs/final.yaml</code> reproduces the full run
Environment	Pinned <code>requirements.txt</code> ; tested on NVIDIA T4 (Colab) and local CUDA 12
Checkpoints	EMA + raw weights saved every N epochs; resumable via <code>--resume</code>
Sampling	<code>python sample.py --ckpt ... --type fire --stage base --style sprite</code>

The mappings, splits, training run, and sampling grids shown in this report are produced by scripts checked into the repository so that results can be regenerated without undocumented manual steps.

Course formatting (page budget). The preceding sections—from the abstract through this tools section—constitute the **main report**. Per course instructions, keep this main portion to **at most 20 pages** when compiled (title page counts toward that total unless your instructor specifies otherwise). The main portion ends on page **??**. If the main body grows too long, move bulky tables, extra result grids, contact sheets, or per-metric breakdowns into Appendix B below rather than expanding the core narrative.

A References

- [1] Ho, J., Jain, A., and Abbeel, P. *Denoising Diffusion Probabilistic Models*. NeurIPS, 2020. <https://arxiv.org/abs/2006.11239>
- [2] Nichol, A., and Dhariwal, P. *Improved Denoising Diffusion Probabilistic Models*. ICML, 2021. <https://arxiv.org/abs/2102.09672>
- [3] Dhariwal, P., and Nichol, A. *Diffusion Models Beat GANs on Image Synthesis*. NeurIPS, 2021. <https://arxiv.org/abs/2105.05233>
- [4] Rombach, R., Blattmann, A., Lorenz, D., Esser, P., and Ommer, B. *High-Resolution Image Synthesis with Latent Diffusion Models*. CVPR, 2022. <https://arxiv.org/abs/2112.10752>
- [5] Karras, T., Aittala, M., Aila, T., and Laine, S. *Elucidating the Design Space of Diffusion-Based Generative Models*. NeurIPS, 2022. <https://arxiv.org/abs/2206.00364>
- [6] Ho, J., and Salimans, T. *Classifier-Free Diffusion Guidance*. NeurIPS Workshop, 2021. <https://arxiv.org/abs/2207.12598>
- [7] Saharia, C., Chan, W., Saxena, S., et al. *Palette: Image-to-Image Diffusion Models*. SIGGRAPH, 2022. <https://arxiv.org/abs/2111.05826>
- [8] Ronneberger, O., Fischer, P., and Brox, T. *U-Net: Convolutional Networks for Biomedical Image Segmentation*. MICCAI, 2015. <https://arxiv.org/abs/1505.04597>
- [9] Perez, E., Strub, F., de Vries, H., Dumoulin, V., and Courville, A. *FiLM: Visual Reasoning with a General Conditioning Layer*. AAAI, 2018. <https://arxiv.org/abs/1709.07871>
- [10] Isola, P., Zhu, J.-Y., Zhou, T., and Efros, A. A. *Image-to-Image Translation with Conditional Adversarial Networks (pix2pix)*. CVPR, 2017. <https://arxiv.org/abs/1611.07004>
- [11] Hugging Face. *Diffusers: State-of-the-art diffusion models for image and audio generation in PyTorch*. <https://github.com/huggingface/diffusers>
- [12] PokeAPI. <https://pokeapi.co/>
- [13] msikma. *pokesprite*. <https://github.com/msikma/pokesprite>
- [14] Project Pokemon. Sprite and asset resources. <https://projectpokemon.org/>
- [15] Pokemon Database. National Pokedex and image resources. <https://pokedexdb.net/pokedex/national>

B Supplementary tables and figures

Use this appendix section for material that supports the main report but need not appear in the 20-page core: for example full-page sampling grids (unconditional, type- or style-conditioned strips, evolution chains, GAN-vs-diffusion comparisons), extended quantitative tables (per-style FID, full ablation grids), or additional mapping contact sheets. The main body should retain only the figures and tables needed to follow the argument; move overflow content here and cite it explicitly from the main text (e.g., “see Appendix B”).